EXPERIMENT 12: ECHO AND CHAT CLIENT–SERVER USING TCP/UDP SOCKETS

AIM

a) To implement an echo client–server using TCP/UDP sockets.
b) To implement a chat client–server using TCP/UDP sockets.

THEORY

A socket is an endpoint for communication between programs over a network. TCP is connection-oriented and provides reliable, ordered data delivery. UDP is connectionless and sends independent datagrams without guaranteeing delivery or order.

ECHO CLIENT–SERVER

In an echo application, the server sends each received message back to the client.

TCP Echo Server (save as `echo_tcp_server.py`)

```
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(("127.0.0.1", 5000))
s.listen(1)
print("TCP echo server waiting...")
conn, addr = s.accept()
with conn:
    print("Connected:", addr)
    while True:
        data = conn.recv(1024)
        if not data:
            break
        conn.sendall(data)
conn.close()
s.close()
```

TCP Echo Client (save as `echo_tcp_client.py`)

```
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect(("127.0.0.1", 5000))
while True:
    msg = input("Message (quit to stop): ")
    if msg.lower() == "quit":
        break
    s.sendall(msg.encode())
    print("Echo:", s.recv(1024).decode())
s.close()
```

Procedure: Run the server first, then run the client in another terminal. Enter a message and verify that the server returns the same text.

Sample output: Client: `Message: Hello` → `Echo: Hello`.

UDP Echo Server (save as `echo_udp_server.py`)

```
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.bind(("127.0.0.1", 5001))
print("UDP echo server waiting...")
while True:
    data, addr = s.recvfrom(1024)
    print("Received:", data.decode())
    s.sendto(data, addr)
```

UDP Echo Client (save as `echo_udp_client.py`)

```
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
while True:
    msg = input("Message (quit to stop): ")
    if msg.lower() == "quit":
        break
    s.sendto(msg.encode(), ("127.0.0.1", 5001))
    data, _ = s.recvfrom(1024)
    print("Echo:", data.decode())
s.close()
```

Procedure: Run the UDP server, then the UDP client in separate terminals. Send a message and observe the echoed response.

Sample output: Client: `Message: Network Lab` → `Echo: Network Lab`.

Observation: TCP uses a connection established with `connect()`/`accept()`; UDP exchanges datagrams using `sendto()` and `recvfrom()`.

CHAT CLIENT–SERVER

A chat application allows two endpoints to exchange messages. This simple TCP example sends one message at a time between a client and server.

TCP Chat Server (save as `chat_tcp_server.py`)

```
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(("127.0.0.1", 5002))
s.listen(1)
print("Chat server waiting...")
conn, addr = s.accept()
print("Connected:", addr)
with conn:
    while True:
        data = conn.recv(1024)
        if not data:
            break
        print("Client:", data.decode())
        reply = input("You: ")
        conn.sendall(reply.encode())
        if reply.lower() == "bye":
            break
s.close()
```

TCP Chat Client (save as `chat_tcp_client.py`)

```
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect(("127.0.0.1", 5002))
while True:
    msg = input("You: ")
    s.sendall(msg.encode())
    if msg.lower() == "bye":
        break
    data = s.recv(1024).decode()
    print("Server:", data)
    if data.lower() == "bye":
        break
s.close()
```

Procedure: Start the server, start the client in another terminal, exchange messages, and type `bye` to end the session.

Sample exchange: Client sends “Hello”; server receives “Hello” and replies “Hi”; client displays “Server: Hi”.

UDP Chat Server (save as `chat_udp_server.py`)

```
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.bind(("127.0.0.1", 5003))
print("UDP chat server waiting...")
while True:
    data, addr = s.recvfrom(1024)
    msg = data.decode()
    print("Client:", msg)
    if msg.lower() == "bye":
        s.sendto(b"bye", addr)
        break
    reply = input("You: ")
    s.sendto(reply.encode(), addr)
    if reply.lower() == "bye":
        break
s.close()
```

UDP Chat Client (save as `chat_udp_client.py`)

```
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server = ("127.0.0.1", 5003)
while True:
    msg = input("You: ")
    s.sendto(msg.encode(), server)
    data, _ = s.recvfrom(1024)
    reply = data.decode()
    print("Server:", reply)
    if msg.lower() == "bye" or reply.lower() == "bye":
        break
s.close()
```

Procedure: Run the UDP server and then the UDP client. Exchange messages; type `bye` to finish.

Sample exchange: Client: “Hello” → Server displays “Client: Hello”; server replies “Hi” → client displays “Server: Hi”.

OBSERVATIONS

1. TCP is reliable and connection-oriented; UDP is connectionless and has lower overhead.
2. Echo programs return the received message unchanged.
3. Chat programs allow the client and server to exchange user-entered messages.

RESULT

Echo and chat client–server applications were implemented using TCP and UDP sockets in Python, and message exchange was verified.